Od surowych danych do sprawdzonej tabeli

Czyszczenie • transformacje • imputacja • testy jakości

Cel: zbudować proces, który można powtórzyć i którego wynik da się sprawdzić.

Plan procesu: kolejność ma znaczenie

01

PUNKT STARTOWY

rozmiar, typy, duplikaty

02

NAPRAWA

teksty, daty, zakresy

03

BRAKI

flagi, mediany, kategorie

04

WERYFIKACJA

testy, raport, eksport

Po każdej większej zmianie sprawdzamy liczbę rekordów, kategorii i braków.

Odczyt i kopia: nie niszcz źródła

PRZYKŁAD SKŁADNI

```
import pandas as pd
import numpy as np

raw = pd.read_csv("sprzedaz.csv")
work = raw.copy()
print(work.shape)
```

CO ROBI

Wczytujemy CSV do DataFrame; copy() tworzy niezależny zbiór roboczy.

KIEDY UŻYWAĆ

Zanim zmienimy wartości lub typy kolumn.

UWAŻAJ

Ścieżka do pliku zależy od katalogu uruchomienia notebooka.

Punkt odniesienia: słownik miar

PRZYKŁAD SKŁADNI

```
start = {
    "rekordy": len(raw),
    "kolumny": raw.shape[1],
    "unikalne_id": raw["transakcja_id"].nunique(),
    "kopie": int(raw.duplicated().sum())
}
print(start)
```

CO ROBI

Słownik zbiera nazwane wyniki. nunique() liczy różne identyfikatory.

KIEDY UŻYWAĆ

Przed czyszczeniem i przy końcowym porównaniu.

UWAŻAJ

Wiersze i unikalne ID mogą się różnić; nie musi to być błąd.

Duplikaty: rozpoznanie i usunięcie

PRZYKŁAD SKŁADNI

```
podejrzane = raw[raw.duplicated(keep=False)]
print(podejrzane.sort_values("transakcja_id"))

work = raw.drop_duplicates().copy()
```

CO ROBI

duplicated() wykrywa pełne kopie; drop_duplicates() je usuwa.

KIEDY UŻYWAĆ

Gdy jeden rekord został zapisany więcej niż raz.

UWAŻAJ

To samo ID przy innych kolumnach może opisywać dwa zdarzenia.

Powtarzalność: proces zamknięty w funkcji

PRZYKŁAD SKŁADNI

```
def clean_sales(data: pd.DataFrame) -> tuple:
    result = data.copy()
    steps = []
    # operacje czyszczące
    return result, steps

clean, log = clean_sales(raw)
```

CO ROBI

Funkcja przyjmuje dane, wykonuje kroki i zwraca wynik oraz historię.

KIEDY UŻYWAĆ

Gdy proces ma być uruchamiany na nowym pliku.

UWAŻAJ

Adnotacje typów ułatwiają czytanie, ale nie walidują danych.

Dziennik transformacji: append()

PRZYKŁAD SKŁADNI

```
history = []
before = len(work)
work = work.drop_duplicates()
removed = before - len(work)
history.append(["duplikaty", removed, "usunięte"])
report = pd.DataFrame(history,
                      columns=["krok", "liczba",
"decyzja"])
```

CO ROBI

Każda operacja otrzymuje opis, liczbę zmian i uzasadnienie.

KIEDY UŻYWAĆ

Kiedy musimy odtworzyć decyzje i skalę czyszczenia.

UWAŻAJ

Kolejność wpisów w dzienniku powinna odpowiadać kolejności operacji.

Tekst: strip() oraz wybór kolumn

PRZYKŁAD SKŁADNI

```
cols = work.select_dtypes(include="object").columns
for col in cols:
    work[col] =
work[col].astype("string").str.strip()
```

CO ROBI

Wybieramy kolumny tekstowe i usuwamy skrajne spacje we wszystkich wierszach.

KIEDY UŻYWAĆ

Gdy „Online ” i „Online” są pozornie dwiema kategoriami.

UWAŻAJ

Nie wszystkie kolumny tekstowe muszą mieć typ object.

Kategorie: lower() i słownik mapujący

PRZYKŁAD SKŁADNI

```
dictionary = {"web": "online", "internet":
"online",
              "store": "stacjonarny"}
work["kanal"] = (work["kanal"]
                 .str.lower()
                 .map(dictionary))
```

CO ROBI

lower() ujednolica litery; map() podmienia dopuszczone warianty.

KIEDY UŻYWAĆ

Gdy źródło używa kilku nazw tego samego kanału.

UWAŻAJ

Wariant nieobecny w słowniku staje się brakiem danych.

Daty: bezpieczne parsowanie

PRZYKŁAD SKŁADNI

```
work["data_zamowienia"] = pd.to_datetime(
    work["data_zamowienia"], errors="coerce"
)
print(work["data_zamowienia"].isna().sum())
```

CO ROBI

Konwertujemy tekst na datę; błędne zapisy zostają oznaczone jako NaT.

KIEDY UŻYWAĆ

Przed badaniem czasu i kolejności zdarzeń.

UWAŻAJ

errors="coerce" nie naprawia formatu: zlicz nowe braki.

Maska błędów: between() oraz ~

PRZYKŁAD SKŁADNI

```
bad_rating = (work["ocena"].notna()
              & ~work["ocena"].between(1, 5))
bad_discount = (work["rabat_proc"].notna()
                & ~work["rabat_proc"].between(0,
100))
print(bad_rating.sum(), bad_discount.sum())
```

CO ROBI

Łączymy reguły zakresu z kontrolą braku danych.

KIEDY UŻYWAĆ

Przy szukaniu ocen 0 lub 9 i rabatów ponad 100%.

UWAŻAJ

~ neguje maskę; każde porównanie otaczamy nawiasami.

loc i np.nan: oznaczamy niepoprawne wartości

PRZYKŁAD SKŁADNI

```
bad = ~work["ocena"].between(1, 5)
work.loc[bad, "ocena"] = np.nan

print(work["ocena"].isna().sum())
```

CO ROBI

loc wybiera konkretne wiersze i kolumnę; np.nan zapisuje brak.

KIEDY UŻYWAĆ

Gdy wartość nie spełnia reguły i nie znamy poprawnej.

UWAŻAJ

Po oznaczeniu błędów jako braki trzeba je uwzględnić w imputacji.

Flaga braku: zapis przed uzupełnieniem

PRZYKŁAD SKŁADNI

```
work["ocena_brakowala"] = (
    work["ocena"].isna().astype(int)
)
print(work["ocena_brakowala"].sum())
```

CO ROBI

Tworzymy kolumnę 0/1 informującą, gdzie brakowało wartości.

KIEDY UŻYWAĆ

Gdy niekompletność danych może mieć znaczenie analityczne.

UWAŻAJ

Flagę trzeba utworzyć przed fillna(), nie po nim.

Imputacja: mediana i fillna()

PRZYKŁAD SKŁADNI

```
typical = work["kwota"].median()
work["kwota"] = work["kwota"].fillna(typical)

print(work["kwota"].isna().sum())
```

CO ROBI

Bierzemy medianę dostępnych kwot i wpisujemy ją w brakujące pola.

KIEDY UŻYWAĆ

Do prostego, uzasadnionego uzupełniania liczb.

UWAŻAJ

Mediana musi być policzona na danych treningowych przy ocenie modelu.

Mediana wewnątrz grup: transform()

PRZYKŁAD SKŁADNI

```
group_median = (work.groupby("rodzaj_dostawy")

["dni_dostawy"].transform("median"))
work["dni_dostawy"] = (
    work["dni_dostawy"].fillna(group_median)
)
```

CO ROBI

Dla każdego rekordu obliczana jest mediana jego grupy.

KIEDY UŻYWAĆ

Gdy czas dostawy zależy od rodzaju usługi.

UWAŻAJ

Pusta grupa może nie mieć mediany: zaplanuj plan B.

Typy po imputacji: round() i astype()

PRZYKŁAD SKŁADNI

```
work["ocena"] = (work["ocena"]
                 .fillna(work["ocena"].median())
                 .round()
                 .astype(int))
```

CO ROBI

Wypełniamy braki, zaokrąglamy, a na końcu zmieniamy typ na int.

KIEDY UŻYWAĆ

Gdy ocena powinna być liczbą całkowitą w ustalonej skali.

UWAŻAJ

astype(int) z nieuzupełnionym NaN spowoduje błąd.

Cechy pochodne: nowe wskaźniki

PRZYKŁAD SKŁADNI

```
work["czas_tyg"] = (work["dni_dostawy"] / 7).round(2)
work["kontakty_razem"] = (
    work["telefony"] + work["wiadomosci"]
)
work["udzial_dostawy_proc"] = (
    100 * work["koszt_dostawy"] / work["kwota"]
).round(1)
```

CO ROBI

Budujemy czas w tygodniach, liczbę kontaktów i udział procentowy.

KIEDY UŻYWAĆ

Gdy pojedyncze pola nie opisują dobrze problemu.

UWAŻAJ

Przed dzieleniem sprawdź zera w mianowniku i moment powstania danych.

Warunek wektorowy: np.where()

PRZYKŁAD SKŁADNI

```
work["dostawa_ekspresowa"] = np.where(
    work["dni_dostawy"] <= 2,
    "tak", "nie"
)
work["zwrot_bin"] = work["zwrot"].map(
    {"nie": 0, "tak": 1}
)
```

CO ROBI

where tworzy kategorię z warunku; map zamienia etykiety na 0 i 1.

KIEDY UŻYWAĆ

Przy tworzeniu flag i kodowaniu zmiennej celu.

UWAŻAJ

Nieznane etykiety po map() staną się brakami.

Porównanie przed i po: wskaźnik zmian

PRZYKŁAD SKŁADNI

```
before = raw.isna().sum()
after = work.isna().sum()
comparison = pd.DataFrame({"przed": before,
                           "po": after}).fillna(0)
print(comparison[comparison["przed"].gt(0)
                | comparison["po"].gt(0)])
```

CO ROBI

Zestawiamy brakujące wartości dla tych samych nazw kolumn.

KIEDY UŻYWAĆ

Po zakończeniu czyszczenia i przed testami.

UWAŻAJ

Mniejsza liczba braków nie gwarantuje poprawności imputacji.

Testy jakości: assert i all()

PRZYKŁAD SKŁADNI

```
assert work["transakcja_id"].is_unique
assert work["ocena"].between(1, 5).all()
assert (work["kwota"] >= 0).all()
assert work["kanal"].isna().sum() == 0
```

CO ROBI

Testy przerywają wykonanie, jeśli choć jeden warunek jest niespełniony.

KIEDY UŻYWAĆ

Po każdym etapie oraz przed zapisaniem pliku.

UWAŻAJ

Test zakresu i test braków to nie to samo.

Audyt cech: nie wszystko wolno modelować

PRZYKŁAD SKŁADNI

```
exclude = ["transakcja_id", "data_zakupu",
           "data_wyplaty_zwrotu"]
target = "zwrot_bin"
features = [c for c in work.columns
            if c not in exclude + [target]]
assert "data_wyplaty_zwrotu" not in features
```

CO ROBI

Lista wyrażeniowa odrzuca pola techniczne i informację z przyszłości.

KIEDY UŻYWAĆ

Przed budowaniem modelu przewidującego zwrot.

UWAŻAJ

Każda cecha musi być znana w momencie prognozy.

Wykres kontroli procesu

PRZYKŁAD SKŁADNI

```
fig, ax = plt.subplots(figsize=(8, 4))
metrics = pd.Series({"surowe": len(raw),
                     "czyste": len(work),
                     "kopie": raw.duplicated().sum()})
metrics.plot(kind="bar", ax=ax)
ax.set_ylabel("Liczba")
plt.tight_layout()
plt.show()
```

CO ROBI

Tworzymy serię miar i prosty wykres słupkowy.

KIEDY UŻYWAĆ

Przy krótkim raporcie skali zmian dla odbiorcy.

UWAŻAJ

Nie mieszaj na jednym wykresie miar o innych jednostkach.

Eksport i sprawdzenie pliku

PRZYKŁAD SKŁADNI

```
from pathlib import Path
path = Path("sprzedaz_czysta.csv")
work.to_csv(path, index=False,
            encoding="utf-8-sig",
            date_format="%Y-%m-%d")
check = pd.read_csv(path)
print(check.shape)
```

CO ROBI

Zapisujemy DataFrame do CSV i natychmiast wczytujemy wynik.

KIEDY UŻYWAĆ

Gdy gotowe dane mają trafić do następnego etapu.

UWAŻAJ

CSV nie przechowuje pełnej informacji o typach pandas.

Zamknięcie: co musi być sprawdzone?

Dane czy rozumiemy jednostkę obserwacji?

Zmiany czy wiadomo co i ile poprawiono?

Braki czy flagi powstały przed imputacją?

Testy czy wartości, ID i kategorie są poprawne?

Cechy czy nie użyliśmy informacji z przyszłości?

Plik czy ponowny odczyt daje oczekiwany wynik?